

Otorisasi dan Keamanan Sumber Daya: Memahami RBAC, Risiko IDOR, dan Proteksi Endpoint di Go

Durasi: 3 Jam (180 Menit) **Level:** Intermediate (Menengah)

Mengetahui siapa pengguna yang masuk (*Authentication*) barulah langkah awal dari keamanan aplikasi. Setelah identitas pengguna terverifikasi, sistem harus menentukan apa saja tindakan yang boleh dilakukan oleh pengguna tersebut (*Authorization*). Tanpa otorisasi yang ketat, aplikasi Anda akan rentan terhadap eksploitasi hak akses seperti **IDOR (Insecure Direct Object Reference)** di mana pengguna biasa dapat melihat atau memodifikasi data milik pengguna lain.

Modul ini akan membedah perbedaan mendasar antara Autentikasi dan Otorisasi, prinsip kerja RBAC, bahaya celah keamanan IDOR, serta strategi pertahanan defensif untuk mengamankan data pengguna di framework Gin.

Sesi 1: Perbedaan Autentikasi vs Otorisasi & Konsep RBAC (45 Menit)

Keamanan aplikasi web bersandar pada dua konsep penting yang sering kali disalahpahami sebagai hal yang sama:

Kriteria	Autentikasi (<i>Authentication</i>)	Otorisasi (<i>Authorization</i>)
Pertanyaan Inti	<i>Who are you?</i> (Siapakah Anda?)	<i>What are you allowed to do?</i> (Apa yang boleh Anda lakukan?)
Fokus Utama	Memverifikasi identitas pengguna (login).	Memeriksa izin akses pengguna ke sumber daya tertentu.
Metode Umum	Login Sandi, OTP, Kunci SSH, OAuth (Google Login).	RBAC (Role-Based), ABAC (Attribute-Based), ACL (Access Control List).
Hasil Akhir	Pengguna mendapatkan token identitas (JWT/Sesi).	Pengguna diizinkan (<i>Allow</i>) atau ditolak (<i>Deny/HTTP 403</i>).

RBAC (Role-Based Access Control)

RBAC adalah metode pembatasan akses sistem berdasarkan peran (*role*) yang diberikan kepada pengguna di dalam organisasi.

- **Aktor & Izin:**

- **ADMIN** : Memiliki hak penuh untuk membuat, membaca, memperbarui, dan menghapus seluruh data (termasuk kelola user).
- **MERCHANT** : Hanya diperbolehkan mengelola produk miliknya sendiri dan melihat pesanan masuk dari tokonya.
- **CUSTOMER** : Hanya diperbolehkan mencari produk, membuat pesanan, dan mengelola profil pribadinya sendiri.

Sesi 2: Risiko Keamanan IDOR (Insecure Direct Object Reference) (45 Menit)

1. Apa itu IDOR?

IDOR (Insecure Direct Object Reference) adalah jenis kerentanan keamanan akses kontrol di mana aplikasi menggunakan referensi langsung ke objek internal (seperti ID database) tanpa memvalidasi apakah pengguna yang meminta data tersebut memiliki hak kepemilikan atas objek itu.

- **Skenario Serangan:**

1. Budi (User ID 5) membuka faktur pesannya di URL: `/api/v1/orders/102`.
2. Budi iseng mengubah angka di URL menjadi: `/api/v1/orders/103` (milik Andi, User ID 8).
3. Karena server tidak memeriksa kepemilikan, server langsung menampilkan detail pesanan 103 ke Budi. Ini adalah celah keamanan **IDOR** yang tergolong dalam kategori *Broken Object Level Authorization* (OWASP Top 10 API Security).

2. Mengatasi IDOR di Go

Ada dua langkah pertahanan utama untuk mencegah IDOR di tingkat kueri database:

1. **Pengecekan Kepemilikan (Ownership Check)**: Jangan pernah melakukan kueri pencarian hanya berdasarkan ID objek yang dikirim oleh klien. Selalu sertakan ID pengguna aktif yang terautentikasi di dalam klausa `WHERE`.

- **✗ Rentan IDOR:**

```
// Siapa pun bisa melihat order jika tahu ID-nya
db.First(&order, orderID)
```

-  **Aman dari IDOR:**

```
// Memaksa pencarian hanya pada order milik user yang sedang aktif
db.Where("id = ? AND user_id = ?", orderID, currentUserID).First(&order)
```

2. **Menggunakan UUID (Universally Unique Identifier):** Menggunakan ID urut integer (1, 2, 3...) mempermudah penyerang menebak parameter (*enumeration attack*). Menggunakan UUID (f81d4fae-7dec-11d0-a765-00a0c91e6bf6) membuat penyerang tidak bisa menebak ID data lain secara acak. Namun, **UUID saja tidak cukup**; pemeriksaan kepemilikan tetap wajib dilakukan!

Sesi 3: Proteksi Endpoint & Role Check Middleware di Gin (45 Menit)

Untuk menjaga kode handler tetap bersih, pengecekan peran (RBAC) harus dipisahkan ke dalam lapisan middleware khusus.

Implementasi Middleware `RequireRole`

Middleware ini menerima argumen daftar peran yang diperbolehkan mengakses rute. Jika peran pengguna yang aktif (yang diekstrak dari JWT Claims) tidak cocok, middleware akan memutus request.

```
package main

import (
    "net/http"

    "github.com/gin-gonic/gin"
)

// Middleware Otorisasi Peran
func RequireRole(allowedRoles ...string) gin.HandlerFunc {
    return func(c *gin.Context) {
        // Mengambil role dari context Gin (disimpan oleh AuthMiddleware
        // sebelumnya)
        userRoleVal, exists := c.Get("role")
        if !exists {
            c.AbortWithStatusJSON(http.StatusForbidden, gin.H{
                "error": "Izin akses ditolak: Informasi peran tidak
                ditemukan",
            })
            return
        }
    }
}
```

```

        userRole := userRoleVal.(string)

        // Periksa apakah role user terdaftar di list allowedRoles
        isAllowed := false
        for _, role := range allowedRoles {
            if userRole == role {
                isAllowed = true
                break
            }
        }

        if !isAllowed {
            c.AbortWithStatusJSON(http.StatusForbidden, gin.H{
                "error": "Akses ditolak: Anda tidak memiliki izin
untuk mengakses resource ini",
            })
            return
        }

        c.Next()
    }
}

```

Sesi 4: Praktikum Mandiri: REST API Invoice Terproteksi dengan Proteksi IDOR & RBAC (45 Menit)

Mari kita buat program REST API e-commerce terproteksi. Program ini akan melindungi data pesanan (`orders`) dari celah IDOR: pengguna biasa hanya boleh membaca pesannya sendiri, sedangkan Administrator diperbolehkan membaca pesanan siapa pun.

Implementasi Kode (`main.go`)

```

package main

import (
    "errors"
    "fmt"
    "net/http"
    "strconv"
    "strings"
    "time"

    "github.com/gin-gonic/gin"
    "github.com/golang-jwt/jwt/v5"
    "gorm.io/driver/postgres"
    "gorm.io/gorm"

```

```

)

// =====
// 1. DATA MODELS & DATABASE INITIALIZATION
// =====

type User struct {
    ID      uint      `gorm:"primaryKey" json:"id"`
    Email   string     `gorm:"type:varchar(100);uniqueIndex;not null"
json:"email"`
    Role    string     `gorm:"type:varchar(20);default:'USER'" json:"role"` //
"ADMIN" atau "USER"
    CreatedAt time.Time `json:"created_at"`
}

type Order struct {
    ID          uint      `gorm:"primaryKey" json:"id"`
    UserID      uint      `json:"user_id"`
    OrderNumber string    `gorm:"type:varchar(50);uniqueIndex;not null"
json:"order_number"`
    ProductName string    `gorm:"type:varchar(100);not null" json:"product_name"`
    TotalAmount float64   `gorm:"type:numeric(12,2)" json:"total_amount"`
    CreatedAt   time.Time `json:"created_at"`
}

var db *gorm.DB
var JWTKey = []byte("KunciRahasia0torisasiSangatKuat123!")

func initDatabase() {
    dsn := "host=localhost user=postgres password=secret45 dbname=ecommercedb
port=5434 sslmode=disable TimeZone=Asia/Jakarta"
    var err error
    db, err = gorm.Open(postgres.Open(dsn), &gorm.Config{})
    if err != nil {
        panic("Gagal menghubungkan database: " + err.Error())
    }
    _ = db.AutoMigrate(&User{}, &Order{})

    // Seeding data awal jika kosong
    var count int64
    db.Model(&User{}).Count(&count)
    if count == 0 {
        user1 := User{ID: 1, Email: "budi@gmail.com", Role: "USER"}
        user2 := User{ID: 2, Email: "andi@gmail.com", Role: "USER"}
        admin := User{ID: 3, Email: "admin@gmail.com", Role: "ADMIN"}
        db.Create(&user1)
        db.Create(&user2)
        db.Create(&admin)

        order1 := Order{ID: 101, UserID: 1, OrderNumber: "ORD-001",
ProductName: "Keyboard", TotalAmount: 500000}
        order2 := Order{ID: 102, UserID: 2, OrderNumber: "ORD-002",
ProductName: "Mouse", TotalAmount: 300000}

```

```

        db.Create(&order1)
        db.Create(&order2)
    }
}

// =====
// 2. MIDDLEWARES (AUTHENTICATION & RBAC AUTHORIZATION)
// =====

type JWTClaims struct {
    UserID uint   `json:"user_id"`
    Email  string `json:"email"`
    Role   string `json:"role"`
    jwt.RegisteredClaims
}

func AuthMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        authHeader := c.GetHeader("Authorization")
        if authHeader == "" {
            c.AbortWithStatusJSON(401, gin.H{"error": "Header otorisasi
diperlukan"})
            return
        }

        parts := strings.Split(authHeader, " ")
        if len(parts) != 2 || parts[0] != "Bearer" {
            c.AbortWithStatusJSON(401, gin.H{"error": "Format otorisasi
harus Bearer <token>"})
            return
        }

        claims := &JWTClaims{}
        token, err := jwt.ParseWithClaims(parts[1], claims, func(t *jwt.Token)
(interface{}, error) {
            return JWTKey, nil
        })

        if err != nil || !token.Valid {
            c.AbortWithStatusJSON(401, gin.H{"error": "Token kedaluwarsa
atau tidak sah"})
            return
        }

        c.Set("userID", claims.UserID)
        c.Set("role", claims.Role)
        c.Next()
    }
}

func RequireRole(allowedRoles ...string) gin.HandlerFunc {
    return func(c *gin.Context) {
        role, exists := c.Get("role")

```

```

        if !exists {
            c.AbortWithStatusJSON(http.StatusForbidden, gin.H{"error":
"Izin akses ditolak"})
            return
        }

        roleStr := role.(string)
        isAllowed := false
        for _, r := range allowedRoles {
            if roleStr == r {
                isAllowed = true
                break
            }
        }

        if !isAllowed {
            c.AbortWithStatusJSON(http.StatusForbidden, gin.H{"error":
"Akses ditolak: Peran Anda tidak diizinkan"})
            return
        }
        c.Next()
    }
}

// =====
// 3. API ENDPOINTS & IDOR MITIGATION LOGIC
// =====

type CreateOrderInput struct {
    ProductName string `json:"product_name" binding:"required"`
    TotalAmount float64 `json:"total_amount" binding:"required,gt=0"`
}

func main() {
    initDatabase()

    r := gin.Default()

    // Endpoint Simulasi Mendapatkan JWT Token
    r.GET("/api/v1/auth/token/:id", func(c *gin.Context) {
        idStr := c.Param("id")
        id, _ := strconv.Atoi(idStr)

        var user User
        if err := db.First(&user, id).Error; err != nil {
            c.JSON(http.StatusNotFound, gin.H{"error": "User tidak
ditemukan"})
            return
        }

        // Terbitkan token JWT
        claims := JWTClaims{
            UserID: user.ID,

```

```

        Email: user.Email,
        Role: user.Role,
        RegisteredClaims: jwt.RegisteredClaims{
            ExpiresAt: jwt.NewNumericDate(time.Now().Add(1 *
time.Hour)),
        },
    }
    token := jwt.NewWithClaims(jwt.SigningMethodHS256, claims)
    tokenString, _ := token.SignedString(JWTKey)

    c.JSON(http.StatusOK, gin.H{"token": tokenString})
})

api := r.Group("/api/v1")
api.Use(AuthMiddleware())
{
    // POST /api/v1/orders - Membuat Pesanan Baru (UserID otomatis diambil
dari token)
    api.POST("/orders", func(c *gin.Context) {
        var input CreateOrderInput
        if err := c.ShouldBindJSON(&input); err != nil {
            c.JSON(http.StatusBadRequest, gin.H{"error":
err.Error()})

            return
        }

        // Mengambil ID user dari token untuk menjamin otorisasi
        userID := c.MustGet("userID").(uint)
        orderNum := fmt.Sprintf("ORD-%d", time.Now().UnixNano()%100000)

        newOrder := Order{
            UserID:      userID,
            OrderNumber: orderNum,
            ProductName: input.ProductName,
            TotalAmount: input.TotalAmount,
        }

        db.Create(&newOrder)

        c.JSON(http.StatusCreated, gin.H{
            "message": "Pesanan sukses dibuat!",
            "data":   newOrder,
        })
    })

    // GET /api/v1/orders/:id - Detail Pesanan terproteksi dari IDOR
    api.GET("/orders/:id", func(c *gin.Context) {
        idStr := c.Param("id")
        orderID, _ := strconv.Atoi(idStr)

        userID := c.MustGet("userID").(uint)
        role := c.MustGet("role").(string)

```



```

        var order Order

        // Penerapan Logika Proteksi IDOR & Otorisasi Hak Akses
        Khusus:
        // Jika pengguna adalah ADMIN, diperbolehkan melihat pesanan
        siapa pun.
        // Jika pengguna biasa (USER), dipaksa mencocokkan kepemilikan
        userID.

        var err error
        if role == "ADMIN" {
            err = db.First(&order, orderID).Error
        } else {
            err = db.Where("id = ? AND user_id = ?", orderID,
            userID).First(&order).Error
        }

        if err != nil {
            if errors.Is(err, gorm.ErrRecordNotFound) {
                // Disarankan untuk mengembalikan 404 (Not
                Found) daripada 403 (Forbidden)
                // agar penyerang tidak mengetahui keberadaan
                data milik orang lain tersebut.
                c.JSON(http.StatusNotFound, gin.H{"error":
                "Faktur pesanan tidak ditemukan"})
                return
            }
            c.JSON(http.StatusInternalServerError, gin.H{"error":
            err.Error()})
            return
        }

        c.JSON(http.StatusOK, gin.H{"data": order})
    })

    // GET /api/v1/admin/orders - Mendapatkan Seluruh Pesanan (Hanya untuk
    ADMIN)
    api.GET("/admin/orders", RequireRole("ADMIN"), func(c *gin.Context) {
        var list []Order
        db.Find(&list)
        c.JSON(http.StatusOK, gin.H{"data": list})
    })
}

r.Run(":8080")
}

```

Sesi 5: Soal Latihan Praktik E-Commerce (Mandiri)

Selesaikan dua latihan pemeliharaan akses kontrol di bawah ini untuk memperkuat pemahaman Anda mengenai mitigasi risiko IDOR dan arsitektur otorisasi di Go.

Latihan 1: Proteksi IDOR pada Manajemen Alamat Pengiriman (Addresses)

Deskripsi Soal: Setiap pengguna dapat menyimpan beberapa alamat pengiriman barang. Lakukan proteksi ketat agar pengguna lain tidak bisa mengubah atau menghapus data alamat milik pengguna lain.

Kriteria Implementasi:

1. Buatlah tabel `shipping_addresses` dengan kolom `id`, `user_id`, `address_line`, dan `city`.
 2. Sediakan dua endpoint di Gin:
 - `PUT /api/v1/addresses/:id` : Mengubah isi alamat.
 - `DELETE /api/v1/addresses/:id` : Menghapus alamat secara fisik.
 3. Lakukan mitigasi IDOR secara defensif pada kedua endpoint tersebut dengan membatasi kueri update/delete hanya dapat dilakukan jika kolom `user_id` di database cocok dengan `userID` aktif dari token JWT.
 4. Pastikan jika record tidak ditemukan atau bukan milik pengguna tersebut, kembalikan response status 404 Not Found demi alasan keamanan informasi (*information disclosure mitigation*).
-

Latihan 2: Desain Relasi RBAC Dinamis (Tabel Roles dan Permissions)

Deskripsi Soal: Dalam platform besar, hardcoding role di kode program (`RequireRole("ADMIN", "MERCHANT")`) kurang fleksibel. Kita ingin agar daftar hak akses bersifat dinamis yang disimpan di database.

Kriteria Implementasi:

1. Rancang tabel relasi database berikut di GORM:
 - `Role` (ID, Name: contoh "ADMIN", "MERCHANT").
 - `Permission` (ID, Name: contoh "CREATE_PRODUCT", "VIEW_REPORT").
 - Hubungkan keduanya menggunakan relasi Many-to-Many (setiap Role memiliki beberapa Permission).
 2. Buatlah middleware otorisasi baru bernama `RequirePermission(allowedPermission string) gin.HandlerFunc`.
 3. Di dalam middleware tersebut, cari data `user_id` dari database, lakukan *join query* ke tabel `roles` dan `permissions`, lalu validasi apakah pengguna memiliki hak akses tersebut sebelum request diproses.
-

Sesi 6: Soal Ujian Project Akhir: Rancang Bangun REST API HRIS & Payroll System (Ujian Akhir)

Deskripsi Project: Anda diminta untuk membangun sebuah sistem backend REST API untuk **HRIS (Human Resource Information System) & Payroll** mandiri menggunakan bahasa Go, framework **Gin**, ORM **GORM**, dan database **PostgreSQL**. Aplikasi harus dirancang menggunakan arsitektur bersih (**Clean Architecture**), mengamankan data dari serangan **IDOR**, membatasi fitur berdasarkan **RBAC**, serta menjaga kekonsistenan data keuangan menggunakan **Database Transactions**.

Kriteria dan Persyaratan Fitur:

1. Manajemen Akun & Autentikasi (JWT & RBAC)

- **Registrasi & Login:**
 - Setiap karyawan didaftarkan oleh HRD dengan field: `email` (harus diakhiri `@company.co.id` menggunakan custom validator), `password` (hash menggunakan `Bcrypt`), dan `role` (bernilai `"HRD"` atau `"EMPLOYEE"`).
- **Autentikasi Middleware:**
 - Amankan seluruh endpoint sensitif menggunakan JWT Token.
 - Terapkan **Token Blacklist** di database saat user melakukan `POST /api/v1/auth/logout`.
- **Otorisasi RBAC:**
 - Buat middleware `RequireRole` untuk memisahkan fitur HRD (akses admin penuh) dan Employee (akses terbatas).

2. Manajemen Penggajian (Payroll System & Transaksi Database)

- HRD dapat memproses penggajian bulanan melalui endpoint `POST /api/v1/payroll/process` dengan menyertakan list `EmployeeID` dan `Bonus`.
- **Persyaratan Transaksi Database:**
 - Gunakan transaksi database (`db.Transaction`) untuk memproses penggajian.
 - Sistem akan membaca gaji pokok karyawan dari tabel `employees`.
 - Gaji total = gaji pokok + bonus.
 - Sistem harus memotong anggaran divisi (tabel `department_budgets`) milik departemen tempat karyawan bekerja.
 - Gunakan **Row-Level Locking** (`FOR UPDATE`) pada baris budget departemen untuk mencegah race condition.
 - Jika sisa budget departemen **tidak mencukupi**, batalkan seluruh transaksi penggajian (`rollback`) dan kembalikan pesan eror yang informatif.

3. Pengajuan Cuti (Attendance & Proteksi IDOR)

- Karyawan biasa (`EMPLOYEE`) dapat mengajukan cuti melalui `POST /api/v1/leaves`.

- Karyawan dapat melihat riwayat pengajuan cutinya sendiri melalui `GET /api/v1/leaves/:id`.
 - **Mitigasi IDOR:** Terapkan kueri defensif. Karyawan biasa tidak boleh melihat atau mengedit data cuti milik karyawan lain meskipun mereka menebak ID cutinya.
 - HRD diperbolehkan melihat detail cuti seluruh karyawan untuk proses penyetujuan.

4. Kerapian Kode (Clean Architecture & DTOs)

- Pisahkan kode program ke dalam struktur folder yang standar:
 - `domain/` : Definisikan core business entities (murni) dan interface kontrak.
 - `repository/` : Implementasi database GORM.
 - `usecase/` : Tempat berkumpulnya logika bisnis penggajian, cuti, dan autentikasi.
 - `delivery/` : Handler HTTP Gin beserta pemetaan DTO Request/Response.
- Seluruh endpoint wajib mengembalikan format JSON yang konsisten untuk pesan sukses dan error.

Petunjuk Pengumpulan:

1. Buatlah repositori Git baru dengan nama `go-hris-payroll-system`.
2. Tuliskan panduan instalasi dan dokumentasi endpoint API di file `README.md`.
3. Kumpulkan URL repositori GitHub Anda ke Learning Management System (LMS) sebelum batas waktu pengumpulan berakhir.